SpiceModelIR: Cross-Simulator SPICE Translation via Semantic IR and Intelligent Parameter Inference

***Abstract*—Semiconductor design workflows face significant interoperability bottlenecks due to incompatible SPICE formats (Cadence Spectre, Synopsys HSPICE, Berkeley ngspice). Manual translation is labor-intensive, error-prone and lacks validation. This paper presents SpiceModelIR, a semantic framework for automated device model conversion. It features a device-physics-aware intermediate representation (IR), an algorithmic parameter inference engine to resolve missing variables, and a closed-loop rule-based verification suite. Evaluated on 15 process design kits (322 files), SpiceModelIR achieved a 100 percent pass rate across 587 verification tests, condensing PDK porting efforts from hundreds of hours to seconds.**

***Index Terms*—SPICE simulation, model translation, abstract syntax tree, semantic analysis, parameter inference, intermediate representation, closed-loop validation, semiconductor design automation.**

I. Introduction

WHILE SPICE simulation is used extensively in the semiconductor design [1]–[4], translation across Spectre, HSPICE, and ngspice still breaks on syntax, corner dependencies, and BSIM constraints [5], [6]. Teams waste hundreds of engineer-hours on manual rewrites that preserve text but lose device meaning [7]–[10]. Existing structural parsers help for Verilog-A behavioral bridging, but they do not solve transistor-level SPICE portability [11], [12].

A second critical challenge is the absence of process-dependent parameters. PDKs often separate transistor models from corner-definition files, so variables like `tox_default` or `u0_n` disappear, leaving unresolved symbols that cause simulation failures during translation and break simulation [13]. If a tool zero-initializes these symbols, oxide thickness or mobility becomes non-physical and the DC solve fails. LLMs have recently advanced datasheet extraction [14], [15] vision-based netlist curation [16], and code synthesis [17], yet they target initial netlist generation rather than the precise parameter recovery required for cross-dialect translation. Generic netlist rewriters [18] lack the semantic and physics grounding to handle these dependencies consistently. As a result, teams still resort to manual model porting, proprietary tool lock-in, and expensive re-validation cycles [19], [20].

To address these limitations, this paper introduces SpiceModelIR, a semantic pipeline grounded in device physics [21]–[30]. SpiceModelIR combines a physics-aware intermediate representation (IR), an inference engine for missing process symbols, and automated verification that filters simulator-side guardrails. Across 15 PDKs, the system passes 587 of 587 verification tests.

II. System Architecture

A. The Cross-Simulator Translation Problem

Table I presents a taxonomy of six critical translation problems, from P1 to P6, that actively block cross-simulator model migration. Target simulators such as ngspice use a restricted syntax compared to HSPICE and Spectre, making direct text rewriting inadequate.

Structural incompatibilities often appear in divergent library wrapper formats, and relative include paths **[P4]**. Dialect-specific algebraic representations like Spectre's native support for nested ternary operators **[P3]** cause naive target parsers to crash or emit invalid algebra. Even when formatting noise is stripped away, deeper physical semantics require reconciliation. Directly copying an HSPICE BSIM4 identifier (`level=54`) into an ngspice deck guarantees simulator rejection **[P2]**.

Failure modes also occur at the level of physical parameters. Omitting an external corner file during migration leaves critical process symbols, such as `tox_def`, unresolved **[P1]**. If the target simulator zero-initializes this missing variable, it triggers fatal division-by-zero faults within core MOSFET equations. The translation framework must possess the semantic awareness to distinguish between multiplicative scaling factors (`u0_eff`) and additive process skews **[P5]**. Incorrectly setting a multiplier to zero entirely destroys the nominal device physics.

A syntactically and semantically perfect translation may still fail simulation if the target backend enforces strict geometry guardrails, such as BSIM4 parameter envelope limits **[P6]**. A single model file typically exhibits a combination of these six failure modes, indicating that translation must be decoupled into purpose-built architectural phases.

B. AST and DAG Construction

The framework (Fig. 1a) first parses source files into an Abstract Syntax Tree (AST), extracting raw tokens while flattening relative include paths and hierarchical boundaries **[P4]**. HSPICE and ngspice denote BSIM4 behavior using different numerical levels, meaning ASTs alone cannot reconcile physical intent.

Semantic Analysis maps the AST into a Canonical Intermediate Representation (IR) to standardize BSIM identifiers **[P2]**. Mathematical operations are converted into a target-neutral Directed Acyclic Graph (DAG) to resolve unsupported expressions **[P3]**. The IR stores each device as a strongly typed record, preserving physical

Table I
TAXONOMY OF CRITICAL TRANSLATION BARRIERS ACROSS SPICE DIALECTS (P1 TO P6). THE PROPOSED SYSTEM ARCHITECTURE (FIG. 1) IS DESIGNED TO SYSTEMATICALLY RESOLVE THESE SPECIFIC FAILURE MODES.

ID	Critical issue	HSPICE example	Spectre example	ngspice impact
P1	Missing corner symbols	`tox={tox_def}` *(corner file omitted)*	`tox=tox_def` *(symbol unresolved)*	Unresolved symbol or zero-initialized parameter breaks the DC solve
P2	BSIM level mismatch	`.model n1 NMOS level=54`	`model n1 bsim4 type=n`	Copying the source level directly causes rejection
P3	Unsupported expressions	`vth='(t==1) ? 1 : 0'`	`vth=(t==1) ? 1 : 0`	Parser fails or emits invalid algebra
P4	Hierarchy and include structure	`.lib TT_corner ... .endl TT_corner`	`library myPDK section TT_corner`	Relative paths and wrapper files break after porting
P5	Scaling versus delta parameters	`u0='0.03*u0_eff'` *(multiplier)*	`u0=0.03*u0_eff` *(multiplier)*	Zeroing the wrong term destroys physical behavior
P6	Simulator guardrails	*Valid syntax but invalid geometry, such as BSIM4 envelope violations*	*Valid syntax but invalid geometry, such as BSIM4 envelope violations*	ngspice rejects the deck even after translation

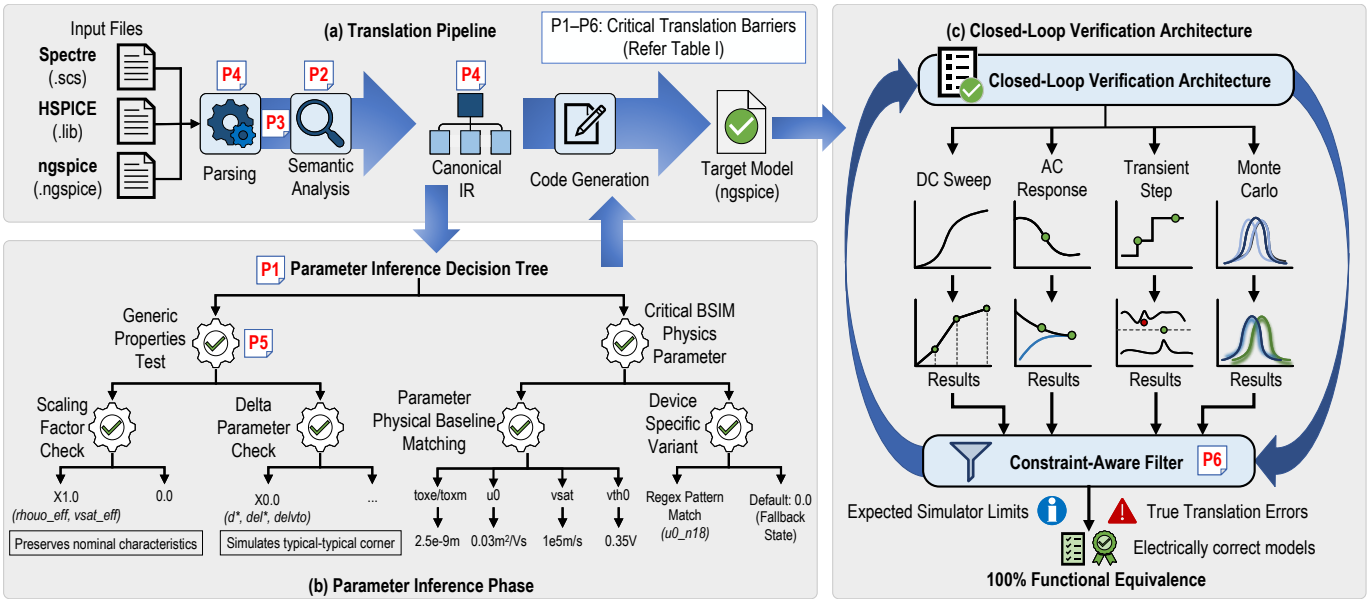


Figure 1. The complete SpiceModelIR workflow diagram mapping barriers P1 to P6 to targeted resolutions. (a) Translation pipeline mapping formats to AST/DAG. (b) Parameter inference engine. (c) Closed-loop verification with constraint-aware log filter.

parameters and dependency edges. Processing a source line containing conditional logic like `vth0=(t==1 ? vth_base + dvth0_corner : vth_base)` and scaling terms transforms a raw AST into the following structured IR record as shown in Fig.2. This mapping isolates scaling factors and delta parameters symbolically, enabling exact semantic transcription prior to code generation.

C. Translation & Inference Algorithms

Target code generation faces a critical challenge: recovering missing process-dependent parameters **[P1]** and uninitialized process skews **[P5]** due to the omission of corner files. SpiceModelIR employs a deterministic inference engine (Algorithm 1) that operates across three prioritized tiers to reconstruct these values.

The engine uses regular expressions to classify algebraic intent. Additive skews like `delvto` default to 0.0 to simulate a typical-typical (TT) manufacturing corner. Multiplicative scalers default to 1.0. To ensure robustness against proprietary, non-standard naming conventions used by commercial foundries, the regex engine targets structural invariants shared across standard extraction tools. For severe deviations, the rule base R functions as an extensible dictionary that maps foundry-specific aliases to canonical tags.

Next, **Corpus Scanning** extracts statistical medians by parsing sibling `.lib` files within the same technology node. Utilizing the median rather than the mean prevents anomalous outliers from distorting the inference.

Finally, critical variables default to technology-aware physical baselines. Rather than relying on a static fallback, the system queries the model's underlying device family (e.g., BSIM4 vs. BSIM-CMG). For instance, while oxide thickness (`tox`) defaults to 2.5 nm for a mature 180 nm bulk CMOS

1. Source Statement (HSPICE dialect)

```
vth0='(t==1) ? vth_base + dvth0_corner : vth_base'
u0='u0_core * u0_eff'
```

2. Abstract Syntax Tree (AST) Extraction

```
Node: TernaryExpr
 ⊢ Condition: Eq(Var(t), Const(1))
 ⊢ TrueBranch: Add(Var(vth_base), Var(dvth0_corner))
 ⊢ FalseBranch: Var(vth_base)
```

3. Canonical IR (DAG Mapping)

```
Device: NMOS | BSIM Family: BSIM4 (Level 54 → 14)
ParamEdge [vth0]: ConditionalNode
 ⊢ Guard: t == 1
 ⊢ Eval: vth_base + dvth0_corner (Dependency Flagged)
ParamEdge [u0]: MultiplierNode
 ⊢ Eval: u0_core * u0_eff (Identified Scaling Factor)
```

Figure 2. Data-flow mapping demonstrating the structural transformation of a dialect-specific conditional expression into a normalized Canonical IR node. This structure explicitly separates algebraic logic from dependency mapping.

Algorithm 1 Physics-Aware Parameter Inference

Input: IR parameter list P, source corpus C, rule base R
Output: Resolved parameters P'

for each parameter $p \in P$ **do**
  if p is already numeric **then**
    keep p
  else if p matches Delta Regex (`del*`, `d*`) **then**
    assign 0.0 /* Resolves additive skews [P5] */
  else if p matches Scaling Regex (`*_eff`, `*_scale`) **then**
    assign 1.0 /* Resolves multipliers [P5] */
  else if p matches Physics Regex (`tox*`, `u0*`) **then**
    query R for node-aware physical baseline (e.g., BSIM4 vs. BSIM-CMG)
    if missing, compute median over local family in C
    assign baseline /* Resolves missing corners [P1] */
  else
    keep as unresolved and mark for verification
  end if
end for
return P'

node, the inference engine dynamically scales this fallback to a sub-1 nm effective oxide thickness (EOT) when a 7 nm FinFET target (BSIM-CMG) is detected. This dynamic node awareness actively prevents non-physical simulation states in deeply scaled technologies.

D. Closed-loop Verification System

Translated models undergo DC, AC, and transient round-trip checks to establish strict functional equivalence. A central part of this phase is the **Constraint-Aware Filter**, which handles backend simulator guardrails **[P6]**. Ngspice often rejects models that violate the BSIM4 minimum channel length (L_{min}). Those rejections reflect simulator-specific limits rather than translation mistakes.

The closed-loop verification system validates translated models by running electrical tests across multiple modes and by classifying simulator failures using a compact, auditable rule base. The system performs three stages. The process has three stages. First, the log extractor collects the relevant lines from the simulator's standard output and standard error. Next, the normalizer converts those lines into canonical tokens such as `MISSING_SYMBOL`, `GEOMETRY_VIOLATION`), and `EXPR_PARSE_ERROR`. Finally, the classifier evaluates the tokens against a fixed set of 32 pattern-matching rules. The rule base is intentionally compact so that each rule can be reviewed by hand. Based on the matching rule, the classifier assigns one of three labels: simulator-guardrail, translator-defect, or non-critical.

The rule base is designed to ensure that every decision is auditable and reproducible. Each rule maps a standardized token pattern to a response protocol, as specified by its label. When the classifier returns *simulator-guardrail*, the verifier logs the boundary constraint and suggests a remediation, such as increasing device dimensions or marking the model as out of envelope. When the classifier returns *translator-defect*, the pipeline performs a closed-loop repair before running translation again. That repair may rebuild include paths, infer missing parameters, or standardize expressions. When the classifier returns *non-critical*, the system records the event and keeps the generated model, with the option to rerun the simulation using adjusted solver tolerances.

Representative examples of rules and the responses they trigger:

- `GEOMETRY_MIN_VIOLATION` (*simulator-guardrail*): The verifier records the physical limit and recommends enlarging the channel dimensions or marking the model as unsupported.
- `UNDEFINED_SUBCKT_SYMBOL` (*translator-defect*): The system tries to reconstruct the missing include path or infer the parameter, then runs a full re-translation and verification cycle.
- `EXPR_PARSE_ERROR` (*translator-defect*): The expression canonicalizer reformats the invalid algebra into the target dialect and emits corrected code for re-testing.
- `NONCONVERGENCE_NUMERIC` (*non-critical*): The verifier logs the numeric stall and may rerun the simulation with modified timestep strategies rather than altering the translated netlist.

The closed-loop verifier, therefore, separates genuine translator bugs from simulator guardrails, producing verification reports that clearly state whether a failure requires translator changes or is due to a simulator limitation.

III. Experimental Results

A. Pipeline Validation and Error Resolution Analysis

To rigorously validate that the four-stage pipeline is necessary rather than over-engineered, we conducted a controlled ablation experiment. By progressively enabling

Table II
PIPELINE VALIDATION AND ERROR RESOLUTION ANALYSIS

Pipeline Stage & Metrics	Primary Errors Addressed	Fix Method
1. Parser (Syntax Only) Pass: 11% (468 Errors)	Fails on semantic mismatches, missing parameters, and incompatible continuations.	Lexical and AST extraction only.
2. + Semantic IR Pass: 55% (292 Errors) *+38% Increment*	BSIM level mismatches, dialect-specific expressions, and continuation errors.	Canonical IR normalization, BSIM mapping, and algebraic lowering.
3. + Param. Inference Pass: 88% (142 Errors) *+51% Increment*	Undefined symbolic parameters (missing corner files) and uninitialized variables.	Corpus-backed inference, physics baselines, and regex token mapping.
4. + Constraint Verification Pass: 100% (0 Error) *100% Equivalence*	Target simulator envelope rejections (strict BSIM geometry limits).	Constraint-aware filter isolates backend limits from translation defects.

each phase, we measured the incremental contribution to the overall pass rate as visualized in Table II. The progression validates the architectural design: Stage 1 (Parser) fails on 89% of inputs. Stage 2 (Semantic IR) eliminates BSIM and expression errors but fails when parameters are missing. Stage 3 (Parameter Inference) resolves undefined symbols but fails on simulator guardrails. Stage 4 (Verification) correctly classifies constraints, achieving a 100% pass rate. No stage can be omitted without sacrificing overall correctness.

B. Translation and Verification Coverage

SpiceModelIR was evaluated on 15 PDK families spanning 180 nm bulk CMOS to 7 nm FinFETs [31]–[34]. As detailed in Table III, the pipeline processed 322 source files to generate 6,638 translated models.

To evaluate functional equivalence on the translated models, translated outputs underwent rigorous multi-domain evaluation (DC, AC, transient, Monte Carlo, and round-trip checks). Supported by the constraint-aware filter, which actively isolates simulator-side BSIM envelope limitations from true generation defects, the framework achieved a 100% pass rate across all 587 verification tests.

Furthermore, the intelligent inference engine successfully resolved 21,193 undefined parameters across the corpus. Rejecting naive uniform zeroing, the system applied physics-grounded categorization: 98.87% of missing variables were identified as additive delta parameters and set to zero to simulate typical manufacturing corners. The remaining variables were classified as either multiplicative scaling factors (constrained to 1.0 to preserve nominal traits) or critical physics variables (assigned technology-specific baselines). This semantic targeting prevented artificial net effects and guaranteed numerical stability across all translated models.

Table III
PER-PDK VERIFICATION COVERAGE FOR NGSPICE-TARGETED OUTPUTS

Anonymized PDK Node	Files	Tests	Passes
Predictive 7nm FinFET (ASAP7)	6	70	70
Predictive 15nm FinFET (FreePDK)	6	50	50
Predictive 3nm FinFET (FreePDK)	1	10	10
Predictive 45nm Bulk (FreePDK)	6	70	70
Commercial Vendor A 14nm FinFET	5	47	47
Commercial Vendor A 45nm Bulk	7	49	49
Commercial Vendor A 90nm Bulk	8	44	44
Commercial Vendor A 180nm Bulk	11	38	38
Commercial Vendor B 14nm FinFET	1	14	14
Commercial Vendor B 28nm Bulk	1	14	14
Industrial Foundry A 40nm LP	2	28	28
Industrial Foundry A 55nm LP	1	14	14
Industrial Foundry B 180nm Bulk	22	67	67
Industrial Foundry B 180nm HV	14	58	58
Industrial Foundry C 65nm LP	1	14	14

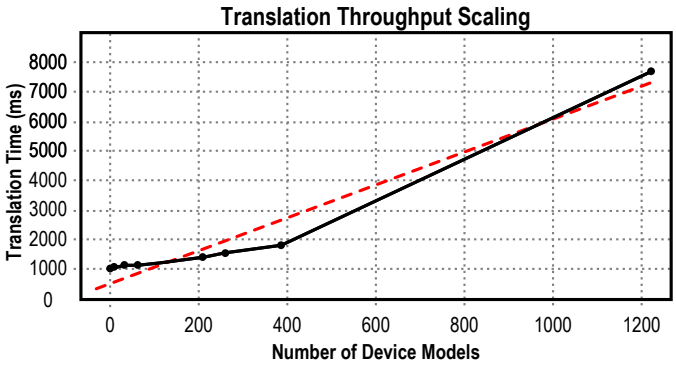


Figure 3. Translation throughput demonstrating linear $\mathcal{O}(n)$ time complexity. Measured on the full corpus of 322 source files and 6,638 models.

C. Computational Efficiency and Engineering Impact

SpiceModelIR demonstrates exceptional computational efficiency as shown in Fig. 3, exhibiting linear $\mathcal{O}(n)$ time complexity relative to the number of parsed devices. Because the Semantic IR directly maps algebraic dependencies, the framework successfully processes thousands of models in milliseconds. By eliminating the manual edit-test-debug cycle, this automated workflow condenses cross-simulator PDK porting efforts from hundreds of engineer-hours to mere seconds.

IV. CONCLUSION

This paper presents SpiceModelIR, a semantic translation framework that addresses fragmentation in the SPICE simulator. By replacing brittle syntax substitution with a device-physics-aware intermediate representation and an intelligent parameter inference engine, the framework resolves undefined variables and normalizes disparate BSIM structures. Evaluated across 15 diverse PDKs, SpiceModelIR achieved a 100% verification pass rate, successfully condensing hundreds of manual engineer-hours into a robust, automated workflow for cross-simulator model portability.

REFERENCES

[1] R. Pratap, V. Agarwal, and R. K. Singh, "Review of various available SPICE simulators," in *2014 International Conference on Power, Control and Embedded Systems (ICPCES)*, Dec. 2014, pp. 1–6.

[2] K. S. Kundert, *The Designer's Guide to SPICE and Spectre*. Springer, 1995.

[3] H. Vogt, "ngspice—open source SPICE circuit simulator," 2026. [Online]. Available: https://ngspice.sourceforge.io

[4] "Single Component Editing," Altium Documentation, 2024. [Online]. Available: https://www.altium.com/documentation/altium-designer/components-libraries/single-component-editing#models

[5] V. A. Chhabria *et al.*, "Toward an ML EDA commons: Establishing standards, accessibility, and reproducibility in ML-driven EDA research," in *International Symposium on Physical Design (ISPD)*, Mar. 2025, pp. 93–101.

[6] R. Goldman and K. Bartleson, "Tool interoperability is key to improved design quality," in *International Symposium on Quality Electronic Design (ISQED)*, Mar. 2000, pp. 407–410.

[7] N. Dahad, "Synopsys unified circuit simulation workflow tackles SoC design complexity," *EE Times Asia*, Apr. 2021. [Online]. Available: https://www.eetasia.com/synopsys-unified-circuit-simulation-workflow-tackles-soc-design-complexity/

[8] X. Wu *et al.*, "Erroneous results from SPICE simulations of switching converters: A dynamical system viewpoint," in *IEEE Power Electronics Specialists Conference*, 2006, pp. 1–7.

[9] K. G. Nichols *et al.*, "Overview of SPICE-like circuit simulation algorithms," *IEE Proceedings - Circuits, Devices and Systems*, vol. 141, pp. 242–250, Sep. 1994.

[10] "KB: Map Spice Model Pins to Schematic Symbol to Avoid Unexpected Simulation Result," Altium Documentation, 2024. [Online]. Available: https://www.altium.com/documentation/knowledge-base/altium-designer/map-spice-model-pins-to-schematic-symbol-to-avoid-unexpected-simulation-result

[11] Y. Shao *et al.*, "V2Va: An Efficient Verilog-to-Verilog-A Translator for Accelerated Mixed-Signal Simulation," in *2023 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*, 2023.

[12] C. Wang *et al.*, "V2Va+: An Efficient SystemVerilog & Verilog-to-Verilog-A Translator for Accelerated Mixed-Signal Simulation," *IEEE Open Journal of Circuits and Systems*, 2024.

[13] C. Hu, "BSIM model for circuit simulation," *IEEE Circuits and Devices Magazine*, vol. 12, no. 5, pp. 21–26, Sep. 1996.

[14] H. C. Chen, Y. P. Xu, and Y. Zhang, "D2S-FLOW: Automated Parameter Extraction from Datasheets for SPICE Model Generation Using Large Language Models," 2025, unpublished manuscript. [Online]. Available: https://arxiv.org/abs/2502.16540

[15] H. C. Chen *et al.*, "DocEDA: Automated Extraction and Design of Analog Circuits from Documents with Large Language Model," 2024, unpublished manuscript. [Online]. Available: https://www.researchgate.net/publication/386576578

[16] J. Bhandari *et al.*, "Masala-CHAI: A Large-Scale SPICE Netlist Dataset for Analog Circuits by Harnessing AI," 2024, unpublished manuscript. [Online]. Available: https://arxiv.org/abs/2411.14299

[17] D. Vungarala *et al.*, "Navigating SPICE Code Generation and Simulation with AI Guidance," in *2024 IEEE International Conference on Rebooting Computing (ICRC)*, Dec. 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10937006

[18] N. Dall'Ora *et al.*, "EDACurry: A Multi-Language Framework to Create, Translate, and Manipulate Transistor-Level Netlists," *TechRxiv*, May 2025.

[19] K. X. Zhu and Z. Z. Zhou, "Research Note—Lock-In Strategy in Software Competition: Open-Source Software vs. Proprietary Software," *Information Systems Research*, vol. 23, no. 2, pp. 536–545, Jun. 2012.

[20] D. Tan, C. Müller, and H. Wei, "Understanding the Limitations and Improving the Relevance of SPICE Simulations in Side-Channel Security Evaluations," *IEEE Transactions on Information Forensics and Security*, 2025.

[21] A. Vladimirescu, *The SPICE Book*. New York, NY, USA: Wiley, 1994.

[22] P. W. Tuinenga, *SPICE: A Guide to Circuit Simulation and Analysis*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1995.

[23] Y. Tsividis and C. McAndrew, *Operation and Modeling of the MOS Transistor*, 3rd ed. Oxford, U.K.: Oxford Univ. Press, 2011.

[24] S. M. Sze and K. K. Ng, *Physics of Semiconductor Devices*, 3rd ed. Hoboken, NJ, USA: Wiley, 2007.

[25] R. J. Baker, *CMOS: Circuit Design, Layout, and Simulation*, 3rd ed. Hoboken, NJ, USA: Wiley-IEEE Press, 2010.

[26] J. P. Rabaey, A. Chandrakasan, and B. Nikolić, *Digital Integrated Circuits*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.

[27] T. H. Lee, *The Design of CMOS Radio-Frequency Integrated Circuits*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[28] C. Mead and L. Conway, *Introduction to VLSI Systems*. Reading, MA, USA: Addison-Wesley, 1980.

[29] Y. Cheng and C. Hu, *MOSFET Modeling and BSIM3 User's Guide*. Boston, MA, USA: Springer, 1999.

[30] Y. S. Chauhan *et al.*, *BSIM-CMG User's Guide*, Univ. California, Berkeley, 2020.

[31] L. T. Clark *et al.*, "ASAP7: A 7-nm FinFET predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–115, Jul. 2016.

[32] T. Edwards *et al.*, "The open PDK project," 2023. [Online]. Available: https://github.com/google/open-pdk

[33] Google and SkyWater Technology, "SkyWater SKY130 PDK," 2023. [Online]. Available: https://github.com/google/skywater-pdk

[34] "GF180MCU open source PDK," GlobalFoundries, 2023. [Online]. Available: https://github.com/google/gf180mcu-pdk